National University of Computer and Emerging Sciences

Assignment No. 01

Data Science

CS4048

Spring 2026

Deadline: As Per GCR

Submission Instructions:

- This is a group assignment.
- Each group should be exactly 2 members.
- **Plagiarism is strictly prohibited Plagiarism among groups would be checked with tools.**
- **Please do not use any AI tool for direct copy / paste and do not copy from others.**
- **Submission carries no marks, Marks will be awarded based on viva**
- Write a Medium Blog and a LinkedIn Post and submit their links
- Develop the front end, and push all code to GitHub with commits from both team members. Submit Repo Link

1. Overview

Retail supermarket chains operate across multiple cities with varying pricing strategies. Understanding market structure, price dispersion, cross-store competition, and price leadership requires large-scale, structured, and clean data pipelines.

Your objective is to design and implement a production-style data engineering pipeline that collects, cleans, normalizes, validates, and matches product pricing data from multiple supermarket chains operating in Pakistan (e.g., Metro, Imtiaz, Al-Fatah, Chase Up, etc.).

The emphasis of this assignment is on Data Engineering architecture, robustness, entity resolution (product matching), scalable processing, statistical rigor, and interpretation.

2. Instructions

- **Scrape product data from at least THREE supermarket chains.**
- **Scrape at least TWO cities per chain (if available).**
- **Collect a minimum of 500,000 rows of raw product data.**
- **Design modular scraping architecture (separate module per store).**
- **Implement retry logic, logging, pagination handling, and rate limiting.**
- **Store raw, processed, and matched data separately (Raw → Processed → Matched layers).**
- **Normalize product attributes (brand, size, unit, price per unit).**
- **Build deterministic cross-store product identity resolution system.**
- **Ensure minimum 10,000 matched cross-store products.**
- **Perform structured price dispersion and competition analysis.**

3.Detailed Price Dispersion & Market Structure Tasks

3.1 Price Dispersion Metrics (Mandatory)

For each matched product across stores and cities, compute:

- **Mean Price**
- **Median Price**
- **Standard Deviation**
- **Coefficient of Variation (CV = std/mean)**
- **Price Range (max - min)**
- **Interquartile Range (IQR)**
- **Price Spread Ratio (max/min)**
- **Relative Price Position Index per store (store_price / category_mean)**

3.2 Store-Level Aggregated Metrics

For each store and city compute:

- Average category price index
- Median price deviation from market average
- Price Volatility Score (average CV across products)
- Price Leadership Frequency (percentage of times store has lowest price)

3.3 Leader Dominance Index (Mandatory Formula)

Define Leader Dominance Index (LDI) as:

LDI_store = (Number of products where store has lowest price) / (Total matched products)

Additionally compute:
- Weighted LDI (weighted by product popularity or category size)
- Category-wise LDI

3.4 Additional Correlation & Competition Analysis (Mandatory Task)

You MUST compute and interpret:

- Correlation between product size and price dispersion
- Correlation between number of competitors (stores selling same product) and price spread
- Correlation between brand tier (premium vs economy) and price volatility
- City-wise price correlation matrix (Pearson/Spearman)
- Cross-store price synchronization score (correlation of matched prices across stores)

4. Methodology & Tools

4.1 Scraping Layer

Use Python scraping frameworks such as requests + BeautifulSoup, Selenium, Playwright, or Scrapy. Implement: retry logic, exponential backoff, structured logging, pagination automation, and rate limiting.

4.2 Cleaning & Normalization

Standardize units (1L = 1000ml), normalize brand names, extract quantities via regex, compute price per unit, remove duplicates, detect missing values, and handle inconsistent currency formats.

4.3 Entity Resolution (Deterministic Matching)

Use brand extraction, size normalization, token cleaning, and fuzzy matching thresholds. Document precision vs recall tradeoffs and matching accuracy.

4.4 Validation Layer (Automated Checks Required)

- Missing value percentage thresholds
- Duplicate detection
- Unit consistency validation
- Outlier detection (Z-score or IQR method)
- Price sanity checks (e.g., price per unit bounds)

4.5 Storage

Use CSV, Parquet, SQLite, or PostgreSQL. Raw, processed, and matched layers must be stored separately.

5. Constraints

- **Minimum 500,000 total rows in cleaned Pandas DataFrame.**
- **Minimum 3 supermarket chains.**
- **Minimum 2 cities per chain (if available).**
- **Minimum 10,000 successfully matched products.**
- **Pipeline must be fully re-runnable.**
- **All scraping attempts must be logged.**
- **No manual data entry allowed.**

6. Deliverables

- **Complete source code (modular and documented).**
- **Final cleaned dataset (500k+ rows).**
- **Matched cross-store dataset.**
- **Analysis results with interpretation.**
- **Technical report (architecture, challenges, solutions).**